Software Engineering II.1

Gruppenprojekt: Photo Album

Software Engineering in einem sich verändernden Projekt · WS 2026/27 · DHBW Heilbronn · Dr. rer. nat. Ana Nicolaescu

Zielgruppe	Teamgröße	Projektform	Bewertung
Studierende Software Engineering II.1	4 Studierende (28 Studierende → 7 Teams)	Semesterbegleitendes Gruppenprojekt über 7 Blöcke	Projektartefakte/Software 50% Abschlusspräsentation & Diskussion 30 % Aktive Teilnahme 20 %

Leitidee: *Nicht möglichst viel Software entwickeln – sondern eine kleine Software nachvollziehbar spezifizieren, entwerfen, implementieren, testen und weiterentwickeln.*

1 Ausgangssituation

Ein Unternehmen möchte eine Anwendung zur Erstellung und Verwaltung digitaler Fotoalben entwickeln. Die primäre Zielgruppe sind Jugendliche und junge Erwachsene. Nutzer:innen sollen Fotos organisieren und daraus digitale Fotoalben erstellen können.

Die Produktidee befindet sich zu Projektbeginn bewusst in einer frühen Phase. Nicht alle Anforderungen stehen fest. Der Auftraggeber (z.B. die Dozentin) gibt im Verlauf des Projekts Feedback, konkretisiert Anforderungen in Sprechstunden und bringt in Block 6 einen Change Request ein, der Anforderungen, Datenmodell, Architektur und Tests berührt.

Ihre Aufgabe ist deshalb nicht nur, eine funktionierende Anwendung zu programmieren, sondern zu zeigen, dass Sie ein kleines Softwareprojekt systematisch spezifizieren, entwerfen, implementieren, testen und unter veränderten Anforderungen kontrolliert weiterentwickeln können.

2 Lernziele

- Requirements Engineering mit prüfbaren Anforderungen (SOPHIST-Regeln) und Akzeptanzkriterien
- Architektur, Softwareentwurf, Entwurfsmuster und begründete Entscheidungen (ADR)
- Versionsmanagement, Code Review und CI im Team
- Systematische Testfallableitung (Blackbox), Whitebox-Analyse und Coverage-Interpretation
- Wartung, Refactoring und kontrollierte Weiterentwicklung (Change Request)
- Qualitäts-, Release-, Konfigurations- und Risikomanagement
- Reflektierter Einsatz von KI-Werkzeugen und Entwurf KI-gestützter Funktionen

Wichtig: Zusätzliche Features führen nicht zu einer besseren Bewertung. Entscheidend sind die Qualität der Software, der Artefakte und des Software-Engineering-Prozesses – und die Nachvollziehbarkeit Ihrer Entscheidungen.

3 Organisation und Meilensteine

Das Projekt wird in Teams à 3-4 Studierenden bearbeitet (Zuteilung in Block 1). Alle Teammitglieder tragen nachvollziehbar und substanziell bei; die Git-Historie ist Teil der Bewertung. Jeder Block der Lehrveranstaltung endet mit einem prüfbaren Zwischenstand im Repository.

Block	Thema	Artefakt im Repository	Vorgeschlagene Abgabe (Git-Tag, Vorabend des Folgeblocks)
1	Einführung, Vorgehensmodelle	Team, Repository, README, Vorgehensentscheidung	block-1
2	Requirements Engineering	REQUIREMENTS.md v1 (Stakeholder, 6–10 FA, 3–5 NFA, 2 Use Cases, Akzeptanzkriterien), UI-Prototyp, AI_USAGE.md Fall 1	block-2
3	Entwurf & Entwurfsmuster	ARCHITECTURE.md (Schichten, Klassen- und Sequenzdiagramm), ≥ 1 ADR, Projektgerüst	block-3
4	Implementierung & Versionsmanagement	Pflichtfunktionalität, Build, PRs mit Review, CI grün, AGENTS.md, Release v0.1.0	block-4
5	Testen (Blackbox)	TESTING.md mit 2 Verfahren, ≥ 10 Unit- und ≥ 2 Integrationstests, Usability-Protokoll, Eval-Set	block-5
6	Whitebox, Wartung, Change Request	Coverage-Interpretation, CHANGE_REQUEST.md (Impact Analysis), REFACTORING.md, Umsetzung CR → Release v1.1.0	block-6
7	Management & Präsentation	RISK_ANALYSIS.md, Release-Entscheidung, Abschlusspräsentation	Nach block-7; spätestens bis 23. November (final)

Der Stand zum jeweiligen Tag wird bewertet; spätere Verbesserungen fließen in die Endbewertung ein, sofern sie bis zum Tag „final“ vorliegen. Der Tag „final“ ist die verbindliche Abgabe.

4 Rolle des Auftraggebers

- Die Dozentin ist der Auftraggeber. Sie beantwortet Fachfragen in Übungen und Sprechstunden – für alle Teams gleich; Antworten werden nach jedem Block im Moodle veröffentlicht.
- Nicht jede Frage wird beantwortet. „Das entscheiden Sie“ bedeutet: Annahme treffen und in REQUIREMENTS.md unter „Annahmen“ dokumentieren.
- Der Auftraggeber ändert seine Meinung: Feedback nach Block 2/3 und ein Change Request in Block 6 sind Teil des Projekts, nicht Störung.
- Der Auftraggeber bewertet nicht die Optik der Oberfläche, sondern ob die Use Cases bedienbar sind und ob Entscheidungen begründet sind.

5 Initialer Projektauftrag

Entwickeln Sie eine Anwendung, mit der Nutzer:innen digitale Fotoalben erstellen und verwalten können.

5.1 Pflichtfunktionalität

- Album erstellen, anzeigen, umbenennen und löschen
- Fotos zu einem Album hinzufügen, anzeigen und entfernen
- Einem Foto einen Titel bzw. eine Beschreibung geben
- Ungültige Eingaben und definierte Fehlerfälle sinnvoll behandeln (fachliche Fehlermeldung, unveränderter Zustand)

5.2 Geschäftsregeln (Ausgangsstand – werden im Requirements Engineering präzisiert)

- Ein Albumname darf nicht leer sein und hat eine definierte Länge
- Ein Album darf maximal eine definierte Anzahl von Fotos enthalten
- Titel bzw. Beschreibungen besitzen eine definierte Maximallänge
- Ein identisches Foto darf nicht mehrfach demselben Album hinzugefügt werden

Die konkreten Werte (Längen, Anzahl, Definition von „identisch“, erlaubte Dateitypen) erfragen Sie beim Auftraggeber in Block 2 und dokumentieren sie als verbindliche Auftraggeber-Entscheidungen.

6 Technische Rahmenbedingungen

Die Anwendung bleibt bewusst klein und überschaubar. Erwartet werden:

- Java 21
- Ein nachvollziehbarer, reproduzierbarer Build (Maven oder Gradle mit Wrapper) und Dependency Management
- Git mit gemeinsamem Repository, geschütztem main-Branch, Pull Requests und Code Reviews
- Automatisierte Tests (JUnit 5) und eine CI-Pipeline (z. B. GitHub Actions), die Build und Tests bei jedem Pull Request ausführt
- Eine einfache, zweckmäßige Benutzeroberfläche (JavaFX, Swing oder Konsole)
- Lokale Datenhaltung im Dateisystem (Format dokumentiert)

Nicht verpflichtend: Authentifizierung/Benutzerverwaltung, Cloud-Deployment, Docker/Kubernetes, Microservices, externe APIs, Chat- oder Bezahlfunktionen, Datenbanken. Zusätzliche Technologien sind erlaubt, wenn sie fachlich begründet werden (ADR) und die CI ohne manuelle Installation läuft.

7 Requirements Engineering (Block 2)

- Relevante Stakeholder und Systemgrenze (Kontextdiagramm)
- 6–10 funktionale Anforderungen nach SOPHIST-Satzschablone mit ID (FA-xx) und Verbindlichkeit (muss/sollte)
- 3–5 nichtfunktionale Anforderungen mit Qualitätsmerkmal, Messgröße und Bedingung
- Mindestens zwei ausgearbeitete Use Cases (Standard- und Alternativabläufe)
- Prüfbare Akzeptanzkriterien (Given/When/Then, AK-xx.n) für alle muss-Anforderungen
- Ein erster klickbarer oder Papier-Prototyp der Oberfläche
- Abschnitte „Auftraggeber-Entscheidungen“ und „Annahmen“; Abschnitt „KI und Sicherheit“ (auch wenn keine KI-Funktion gebaut wird)

8 Architektur und Entwurf (Block 3)

- Grundlegende Architektur (Schichten, Abhängigkeitsrichtung) mit Begründung
- Komponenten, Verantwortlichkeiten und Schnittstellen (Ports)
- Klassendiagramm (Domäne, Anwendung, Ports) und Sequenzdiagramm für „Foto hinzufügen“ inklusive Fehlerpfad
- Mindestens ein Architecture Decision Record (docs/adr/) mit verworfener Alternative
- Entwurfsmuster nur für ein benanntes Problem; jedes eingesetzte Pattern hat ein ADR. Patterns ohne Problem werden nicht positiv bewertet.
- Entwurf einer optionalen KI-Komponente (Port, Adapter, Guardrails) – die Umsetzung ist freiwillig

9 Implementierung, Git und CI (Block 4)

- Repository-Struktur nach Schichten; Dokumente auf oberster Ebene; .gitignore

- Konventionelle Commit-Nachrichten mit Bezug auf Anforderungs- oder Issue-ID; keine „fix“/„WIP“-Commits auf main
- Jede Änderung über Issue → Branch → Pull Request → Review → CI → Merge; mindestens ein dokumentiertes Code Review je Person
- AGENTS.md mit Regeln für Coding Agents; Prompts, Skills und Modellkonfigurationen liegen versioniert im Repository
- Releases als Git-Tags nach Semantic Versioning mit CHANGELOG.md (v0.1.0 nach Block 4, v1.0.0 Pflichtfunktionalität, v1.1.0 nach Change Request)

10 Testing und Qualitätssicherung (Block 5 und 6)

- Mindestens 10 sinnvolle Unit-Tests mit Normal-, Fehler- und Grenzfällen; Fehlerfälle prüfen den unveränderten Zustand
- Mindestens 2 Integrationstests mit echter Infrastruktur und fachlicher Fragestellung
- Mindestens ein, empfohlen zwei dokumentierte Blackbox-Verfahren (Äquivalenzklassen, Grenzwertanalyse, Entscheidungstabelle, Zustandsmodell) mit Tabelle und Zuordnung zu Testmethoden
- Traceability: Akzeptanzkriterium → Verfahren → Testmethode
- Coverage erhoben (JaCoCo) und je Schicht interpretiert; hohe Coverage allein ist kein Qualitätsnachweis
- Usability-Test protokolliert; Security-Tests für Dateieingaben; Performance-Messung für eine NFA
- Regressionstests sichern Change Request und Refactoring ab
- Ein Evaluation Set für eine (auch nur entworfene) KI-Funktion

11 Change Request (Block 6)

In Block 6 stellt der Auftraggeber einen Change Request, der Anforderungen, Datenmodell, Architektur und Tests berührt. Der Inhalt wird erst in Block 6 bekannt gegeben. Vor der Umsetzung erstellt jedes Team eine Impact Analysis in CHANGE_REQUEST.md:

- Betroffene Anforderungen und Akzeptanzkriterien (mit IDs), neue Anforderungen
- Betroffene Teile von Architektur und Datenmodell; ungültig gewordene ADRs, neues ADR
- Tests, die rot werden (Regression) und Tests, die neu entstehen
- Risiken der Änderung und Aufwandsschätzung je Teil; Reihenfolge der Umsetzung
- Nach der Umsetzung: Rückblick – geschätzter vs. tatsächlicher Aufwand, Überraschungen

Bewertet wird nicht nur das neue Feature, sondern insbesondere der nachvollziehbare und kontrollierte Umgang mit der Veränderung. Ein ehrlich dokumentierter, unvollständig umgesetzter Change Request ist besser als ein vollständiger ohne Analyse.

12 Wartung und Refactoring (Block 6)

Wählen Sie einen Teil Ihrer Anwendung, dessen interne Qualität verbessert werden kann – vorzugsweise einen, der den Change Request erleichtert. Dokumentieren Sie in docs/REFACTORING.md: Ausgangssituation (Klasse, Metrik, Smell mit Nachweis), Problem, Änderung (Schritte, jeder mit grünen Tests), Ergebnis und Absicherung durch Tests. Refactoring-Commits ändern kein Verhalten und sind von fachlichen Änderungen getrennt.

13 Einsatz von KI

KI-Werkzeuge dürfen und sollen als Software-Engineering-Werkzeuge untersucht werden – für Requirements (z. B. „Grill me“-Skill), Reviews, Testgenerierung, Debugging, Refactoring und Dokumentation. Die Verantwortung für das Ergebnis bleibt beim Team. Jedes Team führt

AI_USAGE.md mit mindestens drei (empfohlen: fünf, einer je Block 2–6) substanziellen Einsatzfällen:

Feld	Inhalt
Aufgabe/Ziel	Was sollte erreicht werden, in welchem Block?
Werkzeug/Modell	Produkt und Modellkennung, soweit bekannt; Prompt oder Skill (versioniert im Repository)
Ergebnis	Was hat die KI geliefert? (Kurzfassung oder Link)
Kritische Bewertung	Was war richtig, was falsch, was erfunden? Woran erkannt?
Entscheidung	übernommen / angepasst / verworfen – mit Begründung

Bewertet wird nicht die Menge der KI-Nutzung, sondern die transparente, kritische Bewertung. Mindestens ein dokumentierter, begründet verworfener Vorschlag wird erwartet. Coding Agents dürfen eingesetzt werden; ihre Pull Requests durchlaufen denselben Review-Prozess und tragen das Label „ai-assisted“.

14 Release- und Risikoanalyse (Block 7)

Zum Abschluss beantwortet das Team belegt: Würden Sie die aktuelle Version produktiv freigeben? RISK_ANALYSIS.md enthält mindestens:

- Drei Risiken (mindestens ein Produkt- und ein Projektrisiko) mit Eintrittswahrscheinlichkeit und Auswirkung (Risikomatrix) und Begründung
- Maßnahmen zur Risikoreduktion mit Verantwortlichen
- 4–6 Quality Gates für einen Release mit Beleg (erfüllt/nicht erfüllt)
- Rollout-Plan (Strategie, Migration, Rollback) und ein Abschnitt KI-Governance
- Die Release-Entscheidung: ja / nein / unter Bedingungen – mit Bezug auf Matrix und Gates. „Nein, weil …“ wird nicht schlechter bewertet als „ja“.

15 Abgabe

Das Repository enthält zum Tag „final“ mindestens:

- README.md (Zweck, Build, Run, Team, Versionen)
- REQUIREMENTS.md · ARCHITECTURE.md (+ docs/adr/, docs/design/) · TESTING.md · CHANGE_REQUEST.md · docs/REFACTORING.md · AI_USAGE.md · RISK_ANALYSIS.md
- Quellcode und Tests; Build-Konfiguration mit Wrapper; CI-Konfiguration; AGENTS.md; CHANGELOG.md
- Präsentation als PDF in docs/presentation/
- Git-Tags block-1 … block-6, v0.1.0, v1.0.0, v1.1.0, final

16 Bewertung

Die Modulnote setzt sich aus Projektartefakten/Software (50 %) und Abschlusspräsentation einschließlich Diskussion (30 %) zusammen. Gesamtpunkte = Projektpunkte × 0,5 + Präsentationspunkte × 0,3 + Mitmachen in der Klasse * 0,2; erst die Gesamtpunktzahl wird in eine Note umgerechnet.

16.1 Projektartefakte / Software – 100 Punkte

Bereich	Punkte	Bewertungsschwerpunkte
Requirements Engineering	15	FA/NFA klar und testbar (5) · AK und Use Cases sinnvoll (4) · konsistente Dokumentation (3) · Auftraggeber-Feedback eingearbeitet (3)
Architektur & Entwurf	20	Architektur angemessen (6) · Verantwortlichkeiten, Kohäsion, Kopplung (5) · UML korrekt und hilfreich (4) · Entscheidungen begründet, ADR (5)
Implementierung & Codequalität	20	Pflichtfunktionalität funktioniert (8) · Code verständlich/wartbar (5) · Modularisierung/Abstraktion (3) · Fehlerfälle behandelt (2) · Konventionen (2)
Testing & Qualitätssicherung	25	≥ 10 Unit-Tests (6) · ≥ 2 Integrationstests (4) · Blackbox-Verfahren systematisch (5) · Normal-/Fehler-/Grenzfälle (4) · Coverage interpretiert (3) · Regressionstests (3)
Entwicklungsprozess, Git & CI	10	Historie zeigt Zusammenarbeit (3) · Branches/PRs/Review (3) · CI (3) · Struktur (1)
Weiterentwicklung, Reflexion & KI	10	Change Request analysiert und umgesetzt (3) · Refactoring dokumentiert (2) · Risiken/Release reflektiert (2) · KI kritisch dokumentiert (3)

16.2 Präsentation & Diskussion – 100 Punkte

Kriterium	Punkte	Erwartung
Gliederung, Struktur & roter Faden	15	Eine Geschichte von Anforderung über Entscheidung zu Test und Änderung; Zeit eingehalten
Fachwissen & kritische Reflexion	25	Entscheidungen begründet, Alternativen und Schwächen benannt, Fehleinschätzungen zugegeben
Visualisierung & Demonstration	15	Ein Diagramm, das eine Entscheidung erklärt; Demo max. 90 Sekunden, gezielt
Auftreten, Sprache & Teampräsentation	15	Alle vier sprechen substanziell; adressatengerecht
Diskussion & Fragen	30	Fragen an einzelne Personen; jede Person erklärt Anforderungen, Architektur und Qualitätsmaßnahmen des Gesamtprojekts

Format: 10 Minuten Präsentation (harter Stopp) + 5 Minuten Diskussion in Block 7; maximal 8 Folien. Die Präsentation beantwortet: Wie haben sich die Anforderungen verändert? Welche Architekturentscheidung war wichtig und warum? Welcher Testfall ist interessant und wie hergeleitet? Welche Auswirkungen hatte der Change Request? Wie wurde KI eingesetzt und was wurde verworfen? Würde das Team heute releasen?

17 Was eine sehr gute Leistung auszeichnet

Eine kleine, zuverlässige, gut verständliche Anwendung, deren Anforderungen testbar formuliert, deren Architektur nachvollziehbar begründet und deren Qualität durch systematisch hergeleitete Tests abgesichert ist. Der Change Request wurde analysiert, bevor er umgesetzt wurde. Schwächen der eigenen Lösung werden benannt. KI-Werkzeuge werden transparent eingesetzt, ihre Ergebnisse geprüft – und manchmal verworfen. Aktive Beteiligung während der Vorlesung.

18 Checkliste Abgabe

- ☐ Pflichtfunktionalität implementiert (v1.0.0) und Change Request umgesetzt (v1.1.0)
- ☐ 6–10 FA, 3–5 NFA, 2 Use Cases, Akzeptanzkriterien, Auftraggeber-Entscheidungen dokumentiert

- ☐ Architektur, Klassendiagramm, Sequenzdiagramm, ≥ 1 ADR
- ☐ ≥ 10 Unit-, ≥ 2 Integrationstests; Blackbox-Verfahren und Coverage-Interpretation
- ☐ CHANGE_REQUEST.md mit Impact Analysis und Rückblick
- ☐ REFACTORING.md
- ☐ Git-Historie, PRs mit Reviews, CI grün, AGENTS.md
- ☐ AI_USAGE.md mit ≥ 3 Fällen, davon ≥ 1 verworfen
- ☐ RISK_ANALYSIS.md mit Release-Entscheidung
- ☐ Präsentation gehalten; PDF im Repository;
- ☐ Aktive Beteiligung an Vorlesung